

Agents

Think, act, observe, repeat · video 2:46

This lesson answers one question: how does a model carry out a task that takes several steps? An **agent** is the tool loop of episode 7, run until a goal is reached: **think** (the model picks the next action), **act** (the program runs a tool), **observe** (the result joins the context). Two real runs show how agents fail (confident wrong answers, invented facts) and how **tools that check their inputs and explain errors** let the same model recover.

How to use this guide: master one concept before moving to the next.

1. Watch the whole video once, without stopping.
2. Then take the concepts **in order**. For each one: read the short explanation, look at the picture from the video (▶ is the time in the video), and answer every question in its *Check* without looking anything up.
3. Check your answers in the **Answers** section at the end (page 8).
4. **Pass mark: every question in the check correct.** If you missed one, rewatch that concept's video segment, reread the explanation, and redo only the questions you missed. Tick the concept off below, then move on.

It is strongly suggested that you **master each concept before starting the next one**. Each concept builds on the previous ones.

Concepts in this lesson

CONCEPT	VIDEO SEGMENT	QUESTIONS	MASTERED
1. The agent loop	▶ 0:08–0:36	2	☐
2. Confident, and wrong	▶ 0:36–1:19	2	☐
3. Tools that explain errors	▶ 1:19–1:57	2	☐
4. Cost, code and good practice	▶ 1:57–2:44	2	☐

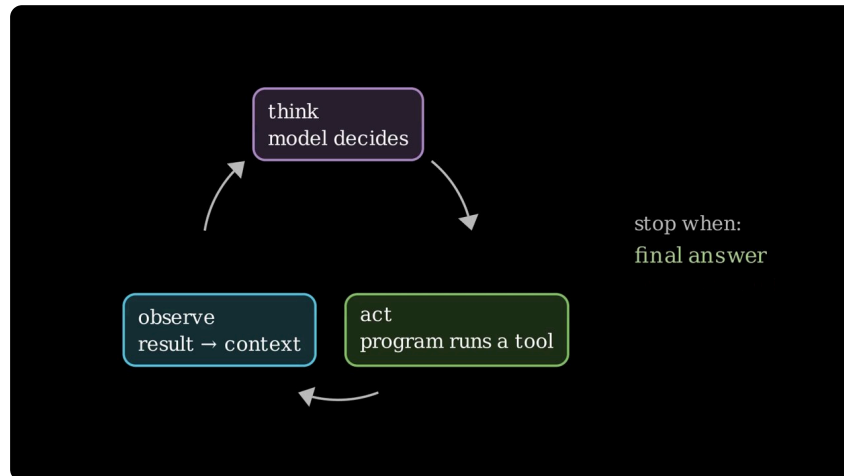
Before you start: episode 7 (tool use) and episodes 1–2 (context and its limits). Code: `code/p08_agent` (downloads Qwen2.5-3B-Instruct, about 6 GB; needs about 7 GB of RAM; runs on a CPU in a minute or two).

CONCEPT 1 OF 4

The agent loop

▶ Video 0:08–0:36

Real tasks take several steps, and each step depends on the last. A model that keeps choosing tools, in a loop, until a goal is reached is called an **agent**. The loop has three parts: **think**, the model decides the next action; **act**, the program runs the tool; **observe**, the result goes back into the context. Repeat until the model gives a **final answer**, or a **step limit** is hit.



▶ 0:35 Think, act, observe, repeat, until a final answer or a step limit.

KEY IDEA

An agent is not a new kind of model: it is a loop around a model that can call tools.

Check 1: The agent loop

Answer all 2 questions before looking at the answers. Pass mark: all correct.

1.1 PUT IN ORDER

Put one turn of the loop in order: *the result is appended to the context · the model decides the next action · the program runs the tool.*

1.2 SHORT ANSWER

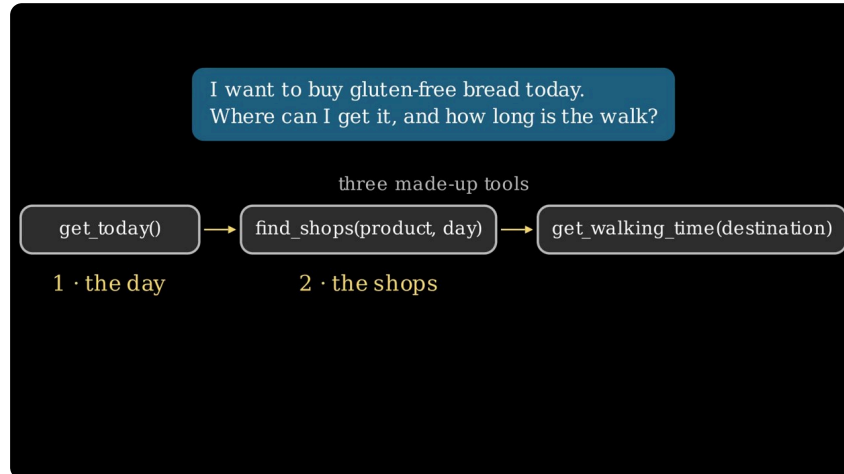
Why does an agent need a step limit?

CONCEPT 2 OF 4

Confident, and wrong

▶ Video 0:36-1:19

The task: “I want to buy gluten-free bread today. Where can I get it, and how long is the walk?” The agent has three made-up tools (get today's date, find shops, get the walking time) and must use them in order: first the day, then the shops, then the walk.



▶ 0:51 The task needs three tools in order: the day, then the shops, then the walk.

On the first try, Qwen2.5-3B called `find_shops` with the day set to “today”, not a weekday name, so the tool found nothing; then it asked for the walk to “shop for gluten-free bread”. Its final answer: no shop sells it today. **Wrong**: Pane Vivo does, on Wednesdays. The even smaller 1.5B model did worse: after getting the day, it simply **invented** two shops, with addresses.

first try · Qwen2.5-3B · tools without checks

```
find_shops(day='today') → shops: []  
get_walking_time('shop for gluten-free bread') → 30 minutes
```

Today there are no shops in Milan that sell gluten-free bread. The walk to the nearest shop selling gluten-free bread would take approximately 30 minutes.

✗ wrong: Pane Vivo sells it on Wednesdays
confident, and wrong

▶ 1:11 First try (3B, tools without checks): a bad argument, an empty result, and a confident wrong answer.

Qwen2.5-1.5B, after getting the day:

```
1. Bread & Butter - Located at Via XX Settembre 20, ... This shop offers various types of bread including gluten-free options.  
2. La Piazza - Situated at Via della Spiga 35, 20124 Milano (Milan), ...
```

invented shops and addresses

▶ 1:18 The 1.5B model, after getting the day, invented two shops with addresses.

KEY IDEA

A tool that silently accepts a bad argument returns a misleading result, and the agent builds a confident answer on it.

Check 2: Confident, and wrong

Answer all 2 questions before looking at the answers. Pass mark: all correct.

2.1 MULTIPLE CHOICE

In the first try, which step caused the wrong final answer?

- A.** `get_today()` returned the wrong day
- B.** `find_shops` was called with `day='today'` and returned an empty list
- C.** The walking time was too long
- D.** The step limit was too low

2.2 TRUE OR FALSE

“If the model's final answer sounds confident and detailed, it was probably based on tool results.”

True ☐ False ☐

CONCEPT 3 OF 4

Tools that explain errors

▶ Video 1:19-1:57

The fix is not in the model: it is in the **tools**. Check every input, and when it is wrong, say **why** and **what to do instead**: “day must be a weekday name such as Monday, not 'today'. Call get_today first.”, “unknown place ... Use a shop name returned by find_shops.”

the fix is in the tools

```
1 def find_shops(product, day):
2     if day not in WEEKDAYS:
3         return {"error": f"day must be a weekday name such as Monday,"
4                     f"not {day!r}. Call get_today first."}
5     ...
```

“day must be a weekday name ... Call get_today first.”
“unknown place ... Use a shop name returned by find_shops.”

▶ 1:32 The fix is in the tools: check every input and say what to do instead.

Second try, same model. Step 1: the same two mistakes, now returned as clear errors. Step 2: it gets the day, Wednesday, but repeats a mistake. Step 3: it checks the day again. Step 4: Wednesday, and the shop list says Pane Vivo. Step 5: 18 minutes. Step 6: the right answer. Six steps, three errors, each fixed by reading an error message.

second try · same model · tools with helpful errors

```
step 1 find_shops(day='today') → error: day must be a weekday name ... call get_today first
step 1 get_walking_time('shop for gluten-free bread') → error: unknown place ... use a shop name
step 2 get_today() → Wednesday
step 2 find_shops(day='today') → error: day must be a weekday name ...
step 3 get_today() → Wednesday
step 4 find_shops(day='Wednesday') → shops: ['Pane Vivo']
step 5 get_walking_time('Pane Vivo') → 18 minutes
```

step 6 Today is Wednesday, and you can buy gluten-free bread at Pane Vivo.
The walk will take approximately 18 minutes.

6 steps · 3 errors · each fixed by reading the error

▶ 1:55 Second try, same model: six steps, three errors, each fixed by reading the error message.

KEY IDEA

Error messages are part of the agent's prompt. Write them for the model: what was wrong, and what to do next.

Check 3: Tools that explain errors

Answer all 2 questions before looking at the answers. Pass mark: all correct.

3.1 SHORT ANSWER

Rewrite this unhelpful tool error for an agent: `{"error": "invalid input"}` (the problem: a date was passed as “next Friday”).

3.2 CALCULATION

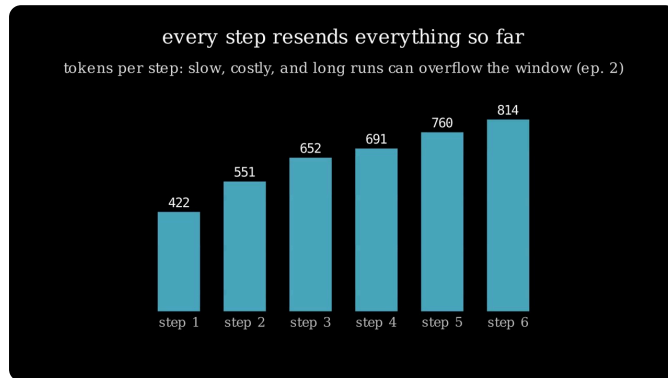
In the second try, how many tool calls did the agent make in total, and how many of them returned errors?

CONCEPT 4 OF 4

Cost, code and good practice

▶ Video 1:57-2:44

Every step resends everything so far: **422 tokens** at the start, **814** at the end. Agents are slow and costly, and long runs can overflow the window (episode 2). In code it is the tool loop with a limit: for each step, generate; no tool call means the final answer; otherwise run every call, append the results, and go around again.



▶ 2:10 Every step resends everything so far: 422 tokens at step 1, 814 at step 6.

- ✓ a step limit
- ✓ tools that check inputs, with helpful errors
- ✓ a small, clear toolset
- ✓ a log of every step
- ✓ a human approves actions with consequences
- ✓ a capable enough model

▶ 2:37 What makes agents work in practice.

To build agents that work: set a **step limit**; make tools **check their inputs**, with helpful errors; keep the toolset **small and clear**; **log** every step; ask a **human** before any action with consequences; and use a model that is **capable enough** for the job.

KEY IDEA

Reliable agents come from the loop around the model: limits, checks, logs and human approval.

Check 4: Cost, code and good practice

Answer all 2 questions before looking at the answers. Pass mark: all correct.

4.1 CALCULATION

The six requests were 422, 551, 652, 691, 760 and 814 tokens. How many tokens did the model read in total?

4.2 HANDS-ON CODE

Try it yourself. In `code/p08_agent/agent.py`: (a) set `MAX_STEPS = 3` and run it. What happens? (b) Set it back to 8 and set `VALIDATE = False`. What is the final answer, and why is it wrong?

```
MAX_STEPS = 3      # (a)
VALIDATE = False   # (b), with MAX_STEPS = 8
```

ANSWERS

Answers and explanations

Mark each answer right or wrong. If a concept has any wrong answer, rewatch its segment (▶) and redo the questions you missed before moving on.

1. The agent loop · rewatch ▶ 0:08-0:36

1.1 the model decides (think) → the program runs the tool (act) → the result is appended (observe). Then the loop repeats with the longer context.

1.2 A model can keep calling tools without ever finishing (looping, repeating mistakes); the limit guarantees the program stops. Every step also costs time and tokens.

2. Confident, and wrong · rewatch ▶ 0:36-1:19

2.1 B. The tool's empty result looked like a real answer ("no shops").

2.2 False. The 1.5B model invented detailed shop names and addresses without any tool returning them.

3. Tools that explain errors · rewatch ▶ 1:19-1:57

3.1 For example: `{"error": "date must be YYYY-MM-DD, not 'next Friday'. Call get_today and compute the date first."}` It names the expected format, echoes the bad value, and suggests the next action.

3.2 7 calls, 3 errors. Step 1: 2 calls (2 errors); step 2: 2 calls (1 error); steps 3, 4, 5: 1 call each.

4. Cost, code and good practice · rewatch ▶ 1:57-2:44

4.1 3,890. $422 + 551 + 652 + 691 + 760 + 814 = 3,890$: the whole context is read again at every step.

4.2 (a) The same first three steps (two errors, get_today, a repeated error, get_today again), then "stopped after 3 steps without a final answer". (b) "Today there are no shops in Milan that sell gluten-free bread. The walk ... would take approximately 30 minutes." Wrong, because find_shops accepted day='today' and returned an empty list, and the walking tool accepted a made-up place. Too few steps stop a recovering agent; tools without checks let it finish confidently with bad data.